

Planning Document A

Client / Advisor Consultation 1

Date of Meeting: 2/8/26

Client/Advisor Name: Eddy

Client/Advisor Email Address: eyang6987@gmail.com

Transcript OR summary of what you and your client discuss.

We narrowed our project topic down. We decided to create an ordering system where people can create their own menu of items to sell, to add items to orders, calculate total and tax, allow order modifications, and save order history. Those are some of the qualities we discussed our program to have.

Key Takeaways

Description of Client Need / Problem

My client's need is to create a digital platform where people can sell their products and to make it accessible for restaurant owners and small business owners looking for a platform to sell on.

Description of Solution

I will use JAVA coding, and might learn other coding languages if they are needed to create a digital platform for people to sell their products.

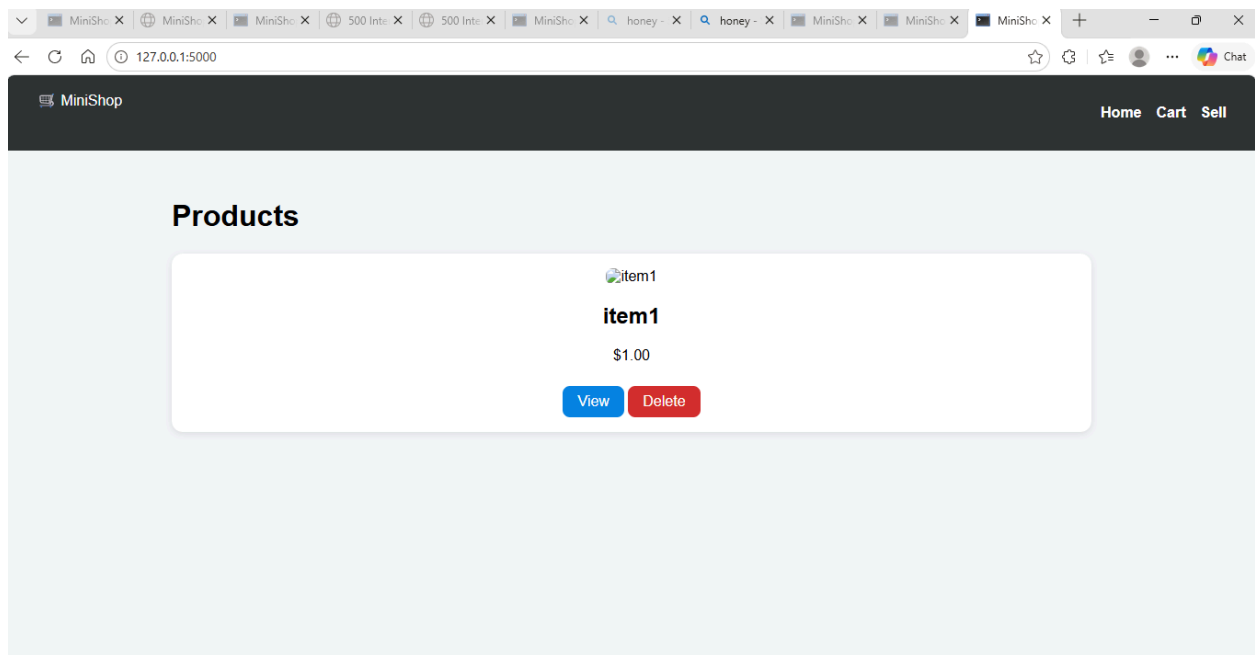
Success Criteria

Success criteria includes the achievement of every quality I of this digital platform I listed before: **to easily create a menu of items to sell, to add items to orders, calculate total and tax, allow order modifications, and save order history.**

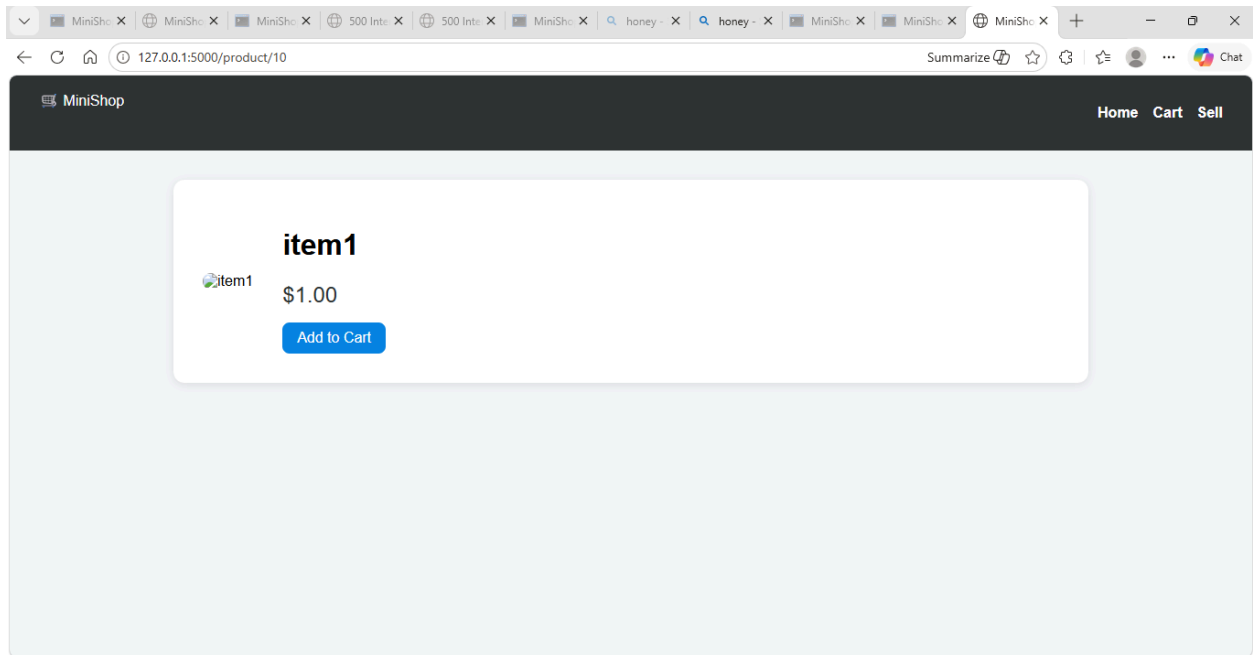
Design Document B

User Interface Design

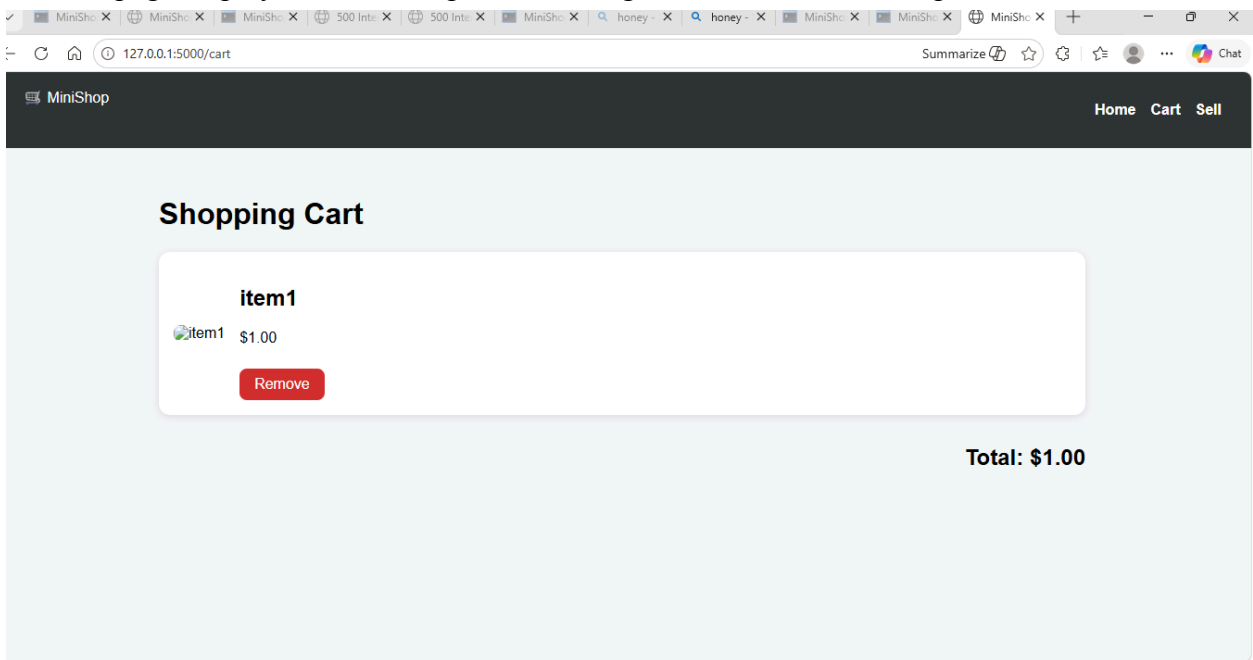
The selling platform I created was designed as a simple and intuitive e-commerce platform with a focus on ease and convenience. The interface is structured using multiple web pages including a home page, product detail page, shopping cart page, and a product listing (selling) page. Each page follows a consistent layout using a shared navigation bar. The home page displays all available products in a grid format, where each product is shown as a card containing an image, name, price, and a button to view more details.



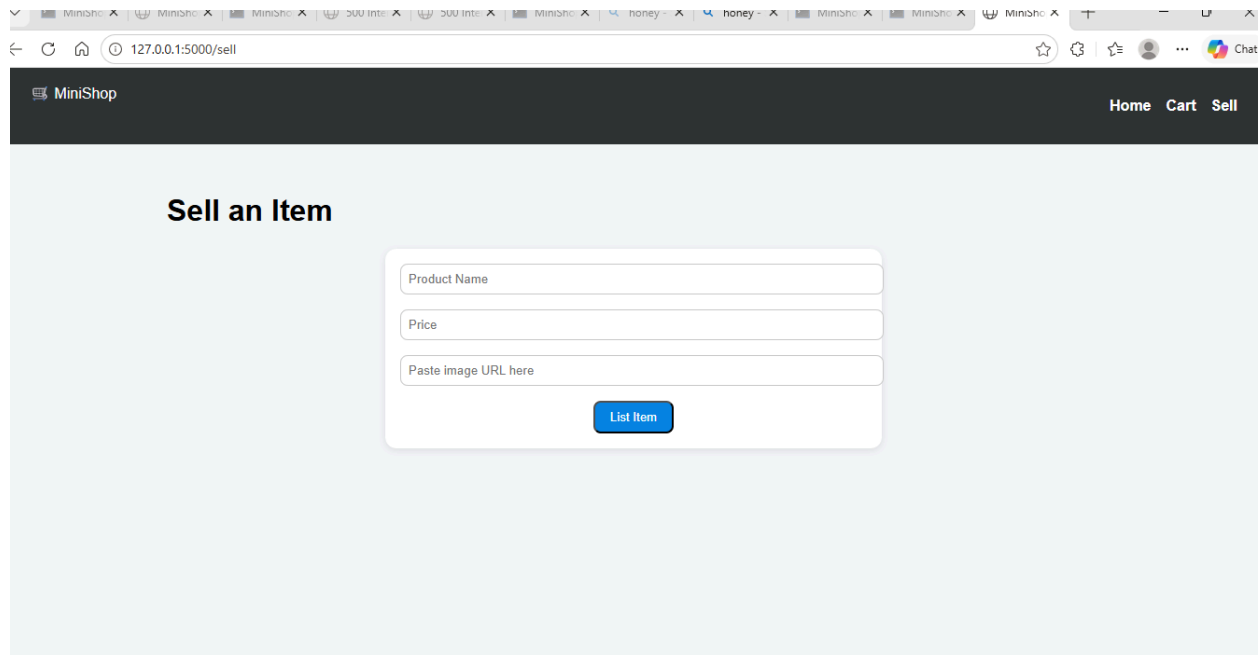
The product detail page allows the user to add it to their cart.



The cart page displays all selected products along with a calculated total price.



Lastly, the sell page provides a form that allows users to add new products by entering a name, price, and image URL.



Program Structure and Methods

The application was built using the Flask framework in Python. Each major feature of the application uses a route which processes user requests and returns the appropriate HTML template.

The `home()` function is responsible for retrieving all products from the SQLite database and displays them on the homepage. The `product_details(pid)` function retrieves a single product based on its ID and displays detailed information. The `add_to_cart(pid)` function inserts a product into the cart table, while the `cart_view()` function retrieves all items in the cart and calculates the total price of all items. The `remove_from_cart(card_id)` function allows users to delete items from their cart.

Furthermore, the `sell()` function allows users to add new products to the database through a form submission. The `delete_product(pid)` function enables removal of products from the system. The `get_db_connection()` function is used to establish a connection with the SQLite database, and the `init_db()` function makes sure that required database tables are created when the application starts.

Data Storage

The application stores its data using an SQLite database. Two main tables are used: a products table and a cart table. The products table stores information about each product including an ID, name, price, and image URL. The cart table stores only product IDs, which are later used to retrieve full product details from the products table.

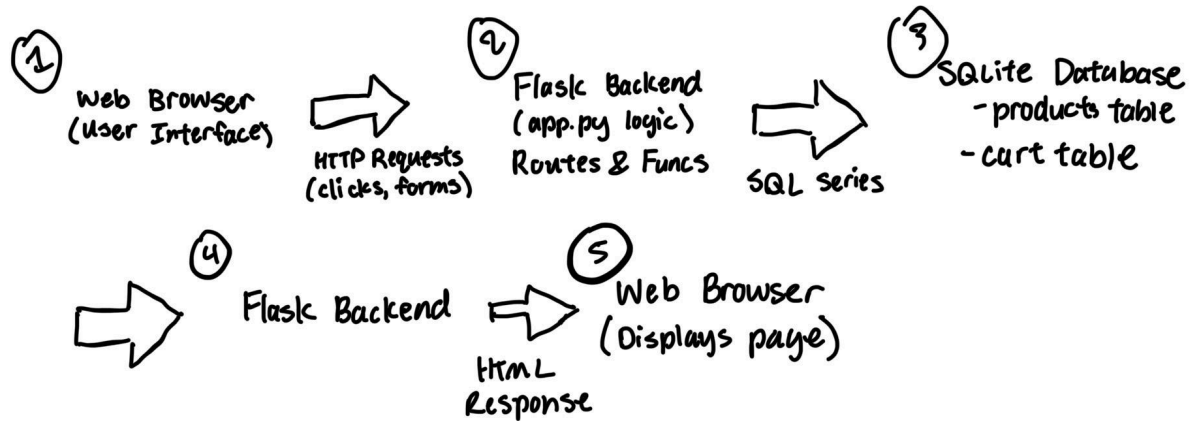
This structure allows storage without duplication of data. When a user adds an item to their cart, only the product ID is stored, and the full product information is retrieved when needed.

Originally, product data was stored in a Python list, but it was replaced with a database system to allow modifiable storage. The database allows products added by users through the sell page to be stored and displayed on the homepage.

Users can add products through the sell page, add items to their cart, and remove items from the cart. These interactions directly adjust the database and immediately affect what is displayed on the website.

Flowchart

The application follows a model where the user interacts with the system through a web browser. When a user performs an action such as viewing products, adding an item to the cart, or submitting a new product, the browser sends a request to the Flask backend. The Flask application acts as the central controller of the system. It processes user requests through defined routes and determines what data is needed. If data is required, Flask communicates with the SQLite database using SQL queries. The SQLite database stores all persistent data, including product information and cart contents. The products table contains details such as product name, price, and image URL, while the cart table stores references to products using product IDs. Once the database returns the required data, the Flask backend processes it and sends it to HTML templates. These templates generate the final web pages, which are then displayed in the user's browser. This continuous cycle allows the application to update content based on their interactions.



- The system retrieves all requests from the cart database table.
- Each cart entry contains a `product_id`.
- For each item in the cart:
 - The program searches the products table for matching products using the `product_id`.
 - The full product information (name, price, image) is retrieved.
- Each product is added to a temporary list representing the cart display.
- The price of each product is added to a total.
- The final list of products and total price are sent to the cart webpage for display.

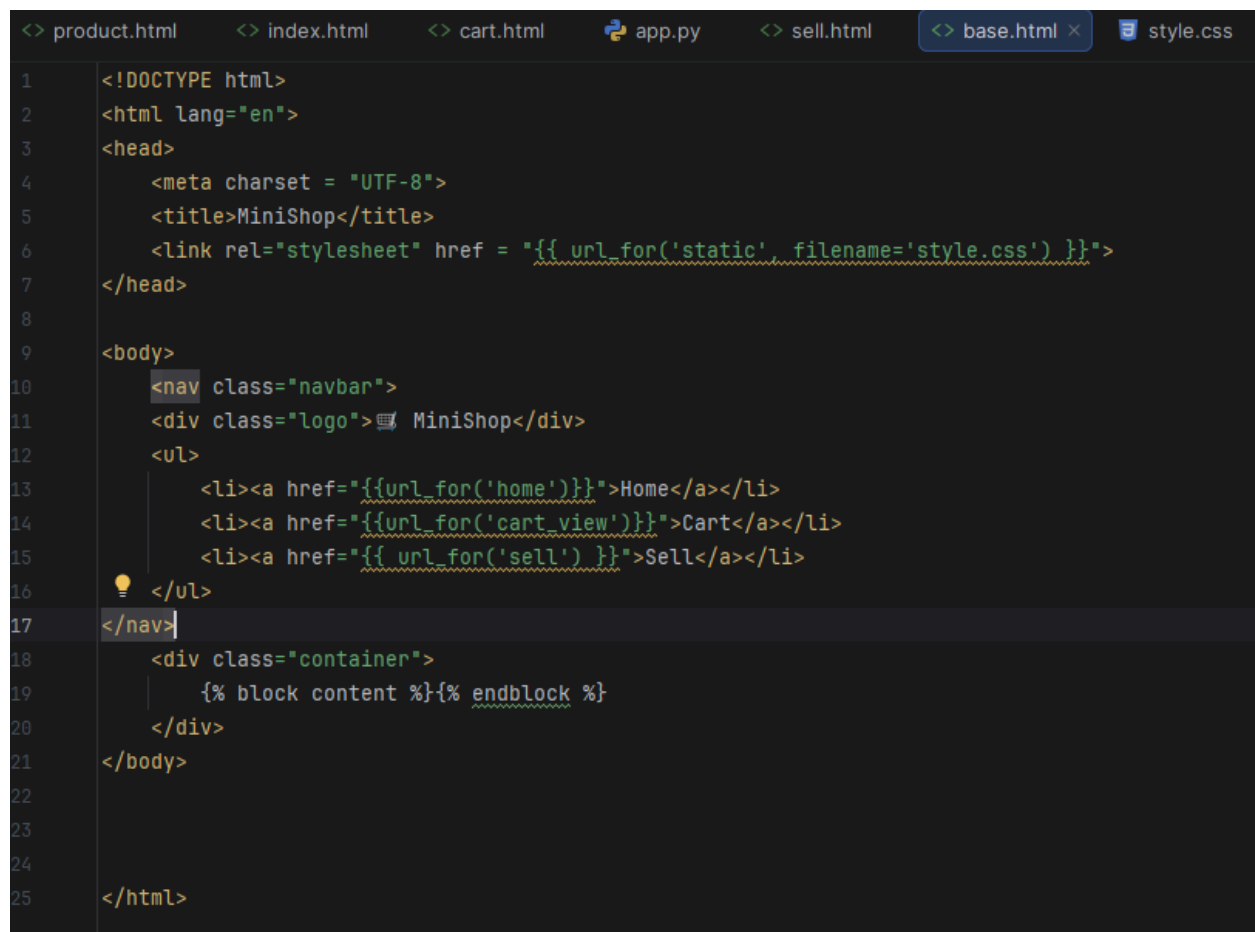
Test Table

Description of Test	Expected Result	Actual Result
Load homepage with products in database	All products are displayed in a grid format with correct images, names, and prices	Products display correctly
View product details page	Clicking "View" shows correct product information	Works as expected
Add product to cart	Product is added and appears in cart page	Works correctly
Remove product from cart	Item disappears from cart and total updates	Works correctly
Add new product using sell form (valid input)	Product is stored in database and appears on homepage	Works correctly
Enter empty product name	Form should not submit or should reject input	Browser prevents submission due to required field
Enter very large price value (e.g. 9999999)	Product is stored and displayed correctly without breaking layout	Displays correctly
Enter very small price (e.g. 0.01)	Product is stored and total calculation remains accurate	Works correctly
Enter invalid image URL	Image fails to load or shows broken image icon	Broken image icon appears (expected behavior)
Delete product from database	Product is removed from homepage and database	Works correctly
Add multiple identical products	All instances appear separately in product list	Works correctly
Access product ID that does not exist	Page should return no product or error-free response	Returns empty/None safely

Implementation Document C

Flask Templates and Navigation System

One of the most important aspects of this application is its user interface, which is built using Flask HTML templates. The main layout is defined in `base.html`, which includes a navigation bar and a place where different pages are loaded using Jinja templating. The navigation bar provides links to the home page, cart page, and sell page, allowing users to move between different parts of the application easily. Each page extends the base template using `{% extends "base.html" %}`, which ensures the same layout across the entire website. This design was made to avoid repeating code. An alternative approach would have been to create separate static HTML pages, but using templates make the application more flexible and easier to maintain.

A screenshot of a code editor with a dark theme. The editor has several tabs at the top: 'product.html', 'index.html', 'cart.html', 'app.py', 'sell.html', 'base.html' (which is the active tab), and 'style.css'. The code in the 'base.html' tab is as follows:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset = "UTF-8">
5     <title>MiniShop</title>
6     <link rel="stylesheet" href = "{{ url_for('static', filename='style.css') }}">
7 </head>
8
9 <body>
10     <nav class="navbar">
11         <div class="logo"> MiniShop</div>
12         <ul>
13             <li><a href="{{ url_for('home') }}">Home</a></li>
14             <li><a href="{{ url_for('cart_view') }}">Cart</a></li>
15             <li><a href="{{ url_for('sell') }}">Sell</a></li>
16         </ul>
17     </nav>
18     <div class="container">
19         {% block content %}{% endblock %}
20     </div>
21 </body>
22
23
24
25 </html>
```

Base.html code.


```
<> product.html  <> index.html x  <> cart.html  app.py  <> sell.html  <> base.html  style.css
1  {% extends "base.html" %}
2  {% block content %}
3  <h1 class="title">Products</h1>
4  <div class="product-grid">
5      {% for p in products %}
6          <div class="card">
7              
8
9              <h2>{{ p.name }}</h2>
10             <p>${{ "%.2f"|format(p.price) }}</p>
11             <a href="{{ url_for('product_details', pid=p.id) }}" class="btn">View</a>
12
13             <a href="{{ url_for('delete_product', pid=p.id) }}" class="btn remove-btn">Delete</a>
14         </div>
15     {% endfor %}
16 </div>
17 {% endblock %}
```

{% extends "base.html" %} is shown at the top of product.html, index.html and cart.html to avoid rewriting the same code.

Product Display (Home page)

The home page is controlled by the home() function in Flask. This function retrieves all product records from the SQLite database using a SQL SELECT query and passes them into the index.html template. Inside the template, a loop (for p in products) is used to generate product 'cards'. Each card displays the product image, name, price, and a button that links to the product detail page. This approach allows the website to automatically update whenever new products are added without manually editing the HTML file. A list-based approach was initially used, but was replaced by a database for more flexible storage and better convenience for updates.

```
@app.route("/")
def home():
    conn = get_db_connection()
    products = conn.execute("SELECT * FROM products").fetchall()
    conn.close()

    return render_template(template_name_or_list="index.html", products=products)
```

home() function sends product information to index.html.

```
<> product.html <> index.html x <> cart.html <> app.py <> sell.html <> base.html <> style.css
1 {% extends "base.html" %}
2 {% block content %}
3 <h1 class="title">Products</h1>
4 <div class="product-grid">
5     {% for p in products %}
6         <div class="card">
7             
8
9             <h2>{{ p.name }}</h2>
10            <p>${{ "%.2f"|format(p.price) }}</p>
11            <a href="{{ url_for('product_details', pid=p.id) }}" class="btn">View</a>
12
13            <a href="{{ url_for('delete_product', pid=p.id) }}" class="btn remove-btn">Delete</a>
14        </div>
15    {% endfor %}
16</div>
17{% endblock %}
```

HTML code in index.html; the sections that allows the website to display image, name and price of products.

Shopping Cart

The shopping cart system is one of the most important features of the application. When a user adds a product to the cart, only the product ID is stored in the cart table in the database. The `cart_view()` function retrieves all cart requests, then uses each product ID to look up full product details from the products table. These details are made into a list that is displayed on the cart page. At the same time, the total price is calculated by going through all cart items and summing up their prices. This design was chosen to avoid duplicates of the same data, since storing full product information in the cart would lead to unnecessary repetition. An alternative would have been storing full product objects, but this would make updates less efficient and more difficult.

```

@app.route("/cart")
def cart_view():
    conn = get_db_connection()

    cart_items_raw = conn.execute('SELECT * FROM cart').fetchall()

    cart_items = []
    for item in cart_items_raw:
        product = conn.execute(
            sql: "SELECT * FROM products WHERE id = ?",
            parameters: (item["product_id"],)
        ).fetchone()

        if product:
            cart_items.append({
                "cart_id": item["id"],
                "id": product["id"],
                "name": product["name"],
                "price": product["price"],
                "img": product["img"]
            })

```

cart_view() function written in [app.py](#)

SQLite Database System

The application uses an SQLite database to store both product and cart data. The database contains two tables: products and cart. The products table stores all product-related information such as name, price, and image URL. The cart table stores only product IDs, which are used to reference products in the products table. The get_db_connection() function is used throughout the application to connect to the database, and the init_db() function ensures that the required tables exist when the application starts. This approach was chosen because SQLite is lightweight, easy to set up, and ideal for smaller applications.

```

def get_db_connection(): 8 usages
    conn = sqlite3.connect(DATABASE)
    conn.row_factory = sqlite3.Row
    return conn

def init_db(): 1 usage
    conn = get_db_connection()

    conn.execute('''
        CREATE TABLE IF NOT EXISTS cart (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            product_id INTEGER
        )
    ''')

    conn.execute('''
        CREATE TABLE IF NOT EXISTS products (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT,
            price REAL,
            img TEXT
        )
    ''')

```

get_db_connection() and init_db() under app.py

Sell Page

The product system allows users to add new products through the /sell route. The user enters a product name, price, and image URL into a form, which is then submitted to the Flask backend using a POST request. The server retrieves the form data using request.form, then inserts a new product into the database using SQL INSERT query. After the product is added, the user is redirected back to the home page where the new product is immediately visible.

```

@app.route(rule="/sell", methods=["GET", "POST"])
def sell():
    if request.method == "POST":
        name = request.form["name"]
        price = request.form["price"]
        img = request.form["img"]

        if not img.startswith("http") and not img.startswith("/static"):
            img = "/static/pictures/" + img |

        conn = get_db_connection()
        conn.execute(
            sql: "INSERT INTO products (name, price, img) VALUES (?, ?, ?)",
            parameters: (name, price, img)
        )
        conn.commit()
        conn.close()

        return redirect(url_for("home"))

    return render_template("sell.html")

```

/sell route written in app.py

Resources Used

The application uses the Flask framework to handle routing, templating, and different kinds of logic that is alien to me. Much of the knowledge I gathered to create this application was from the help of YouTube videos I watched online, consulting with my teacher, and asking clarifying questions using artificial intelligence. Much of the debugging and improving database structure came mostly from meeting with my teacher.

Video D

https://drive.google.com/file/d/18flHOW_oOL4PxD7kFjPE6KJF4nwXuoSb/view?usp=sharing

Final Evaluation and Reflection Document E

Starting from the first item in my success criteria list, I have successfully created a page where users can add items to sell, requiring the user to put in a name, its price, and an image. On the customers' end, I have created a "cart" page where users can modify their orders, adding or removing items from the cart as necessary, hence completing the second and fourth criteria. Once customers are finished with browsing, they can view their calculated total in their shopping cart, therefore I have accomplished the third item of the criteria. I have not gotten to the point where tax can be calculated into the total; I would need to go through certain legal procedures before that occurs. Lastly, customers are able to view their stopping cart anytime they'd like; the items in their cart will be permanent unless removed no matter how much back and forth they do between different pages on the navigation bar.

After consulting with my client, also my father, he is impressed with how well the system works. He believes that there is much to improve, such as getting the images to actually work, and to separate customer and owner pages. There are still many steps left before we can finish the final product.

If I had two more months to work on this project (and I will, especially during summer break), I would first fix some of the errors in the website, the only prominent one would be the working images. From there, I would try to separate customer and owner pages, get everything set up nicely before undergoing the legal processes of this project, such as credit card transfers, establishing a workable website open publicly, and actually coming up with unique products to sell.

From this project, I was able to grasp a firm understanding of how websites online truly work. There are several elements that act together in unity before each masterpiece is created, and I find that especially interesting. I learned how different platforms can communicate with each other, how and where information is stored, and the pathways where information passes through to create a working website. I also handled several foreign coding languages in the process. Though I have worked with HTML before, this project allowed me to dive deeper into this language, and how it worked with Python and CSS. On the other hand, alien languages like Python and CSS are languages I have never worked with before, so getting some experience with these languages is quite a lovely experience.